**Experiment 04**

# Implement packet sniffing using raw sockets in python

## Aim

To implement packet sniffing using raw sockets and decode packet headers.

## Requirements

Linux AF_PACKET for live capture, appropriate privileges, Python 3.10+. Offline decoding tests run on any supported Python platform.

## Procedure / algorithm

1. Open AF_PACKET SOCK_RAW with ETH_P_ALL and bind the assigned interface.

2. Receive Ethernet frames, validate length and unpack MAC addresses and EtherType. Handle one VLAN tag when present.

3. For IPv4, validate version/IHL/total length and display IP addresses, TTL and protocol. Mark fragments instead of treating fragment payload as a complete transport header.

4. Decode TCP ports, sequence, ACK and flags; UDP ports and length; ICMP type/code. Identify ARP by EtherType.

5. Print a fixed number of frames and close the socket. Use --hex to decode the supplied deterministic synthetic fixture offline.

6. Run parser tests for ICMP/TCP/UDP, VLAN, ARP, fragments and truncated frames.

## Complete source / configuration: sniffer.py

```
"""Linux raw Ethernet capture plus a platform-independent frame decoder."""
import argparse
import json
import socket
import struct
from pathlib import Path


def mac(value):
    return ":".join(f"{b:02x}" for b in value)


def decode_frame(frame):
    if len(frame) < 14:
        raise ValueError("Truncated Ethernet header")
    dst, src, ethertype = struct.unpack("!6s6sH", frame[:14])
    result = {"source_mac": mac(src), "destination_mac": mac(dst),
              "ethertype": f"0x{ethertype:04x}"}
    offset = 14
    if ethertype in (0x8100, 0x88a8):
        if len(frame) < 18:
            raise ValueError("Truncated VLAN header")
        tag, ethertype = struct.unpack("!HH", frame[14:18])
        result["vlan"] = tag & 0x0fff
        result["ethertype"] = f"0x{ethertype:04x}"
        offset = 18
    if ethertype == 0x0806:
        result["protocol"] = "ARP"
        return result
    if ethertype != 0x0800:
        result["protocol"] = "Other Ethernet"
        return result
    ip = frame[offset:]
    if len(ip) < 20 or ip[0] >> 4 != 4:
        raise ValueError("Invalid IPv4 header")
    ihl = (ip[0] & 15) * 4
    total = struct.unpack("!H", ip[2:4])[0]
    if ihl < 20 or total < ihl or len(ip) < total:
        raise ValueError("Truncated IPv4 packet")
    protocol = ip[9]
    result.update(source_ip=socket.inet_ntoa(ip[12:16]),
                  destination_ip=socket.inet_ntoa(ip[16:20]),
                  ttl=ip[8], length=total, header_length=ihl,
                  protocol={1: "ICMP", 6: "TCP", 17: "UDP"}.get(protocol, str(protocol)))
    fragment = struct.unpack("!H", ip[6:8])[0]
    if fragment & 0x3fff:
        result["fragmented"] = True
        return result  # Do not interpret fragment bytes as a transport header.
    data = ip[ihl:total]
    if protocol == 1:
        if len(data) < 8:
            raise ValueError("Truncated ICMP header")
        result.update(icmp_type=data[0], icmp_code=data[1])
    elif protocol in (6, 17):
        minimum = 20 if protocol == 6 else 8
        if len(data) < minimum:
            raise ValueError("Truncated transport header")
        sport, dport = struct.unpack("!HH", data[:4])
        result.update(source_port=sport, destination_port=dport)
```

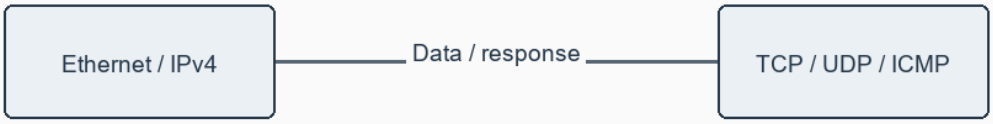
```
        if protocol == 6:
            result.update(sequence=struct.unpack("!I", data[4:8])[0],
                          acknowledgement=struct.unpack("!I", data[8:12])[0],
                          flags=data[13])
        else:
            result["udp_length"] = struct.unpack("!H", data[4:6])[0]
    return result


def capture(interface, count=10, timeout=10):
    if not hasattr(socket, "AF_PACKET"):
        raise OSError("Live AF_PACKET capture requires Linux; decoder tests are portable")
    if count < 1:
        raise ValueError("Count must be positive")
    with socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(3)) as s:
        s.bind((interface, 0))
        s.settimeout(timeout)
        for _ in range(count):
            frame, _ = s.recvfrom(65535)
            try:
                print(json.dumps(decode_frame(frame), sort_keys=True))
            except ValueError as error:
                print("Skipped:", error)


def main():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--interface", default="eth0")
    p.add_argument("--count", type=int, default=10)
    inputs=p.add_mutually_exclusive_group()
    inputs.add_argument("--hex", help="Decode one supplied Ethernet frame in hexadecimal")
    inputs.add_argument("--hex-file", help="Read a hexadecimal Ethernet frame from a file")
    a = p.parse_args()
    if a.hex or a.hex_file:
        value=Path(a.hex_file).read_text() if a.hex_file else a.hex
        print(json.dumps(decode_frame(bytes.fromhex(value)), indent=2, sort_keys=True))
    else:
        try:
            capture(a.interface, a.count)
        except (PermissionError, OSError, socket.timeout) as error:
            raise SystemExit(str(error))


if __name__ == "__main__":
    main()
```

### Diagram: headers




IPv4 IHL determines the transport offset; fragments are not decoded as full headers.

## Run

From the repository root unless a cd command is shown.

```
sudo python3 Exercise_04/sniffer.py --interface eth0 --count 10
python Exercise_04/sniffer.py --hex-file Exercise_04/outputs/fixture.hex
```

## Actual output / verification

```
ACTUAL CLI DECODER RUN ON SYNTHETIC TEST FRAME (not live capture)
{
  "destination_ip": "10.1.1.1",
  "destination_mac": "02:00:00:00:00:01",
  "ethertype": "0x0800",
  "header_length": 20,
  "icmp_code": 0,
  "icmp_type": 0,
  "length": 33,
  "protocol": "ICMP",
  "source_ip": "10.1.1.2",
  "source_mac": "02:00:00:00:00:02",
  "ttl": 64
}

Live AF_PACKET capture requires Linux and was not run on Windows.
```

## Expected output - not an execution record

Live capture prints MAC/IP addresses and protocol fields from traffic reaching the interface. A switched interface does not automatically see every packet in the LAN.

## Observations

Actual output comes from running the decoder on an explicitly synthetic Ethernet/IPv4/ICMP fixture. Linux AF_PACKET live capture was not executed on this Windows host. The separate Wireshark loopback capture must not be presented as this program's live Ethernet output.

## Result

Complete live-capture code is supplied, and the offline decoder/edge cases were verified. Linux live raw capture is pending.